# Manual técnico



Esta guía sirve para instalar, ejecutar, desplegar, diagnosticar y mantener la aplicación, a su vez describe el comportamiento comprobado en app/ y en los scripts PostgreSQL de database, las limitaciones y recomendaciones de operación se identifican como tales, a lo largo del documento.

## Alcance

Incluye el registro, consulta, modificación y eliminación (CRUD) de organizadores, tipos de evento, ubicaciones, recursos y eventos con sus recursos, consultas por fecha, por tipo y de disponibilidad y mecanismos de integridad en la base de datos, no incluye autenticación de usuarios ni control de permisos dentro de la aplicación, ni la detección de choques de horario.

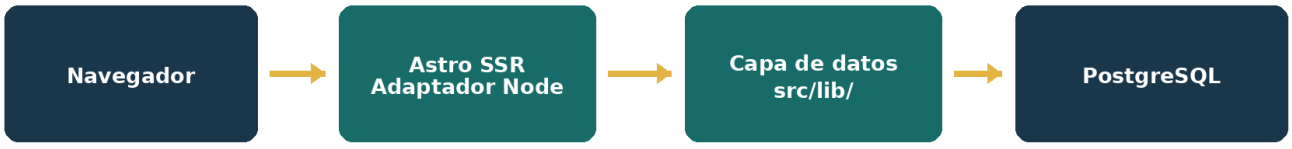
### Tecnologías y entotrno

| Componente | Tecnología | Función |
|---|---|---|
| Motor de base de datos | PostgreSQL 16 | Almacenamiento y cumplimiento de la integridad |
| Cliente SQL | psql | Ejecución de scripts y pruebas |
| Aplicación web | Astro 7 en modo servidor (SSR) con adaptador de Node | Páginas y formularios |
| Acceso a datos | pg (node-postgres), consultas parametrizadas | Comunicación con PostgreSQL |
| Desarrollo | Linux | Entorno de desarrollo |

## Arquitectura

Astro renderiza las páginas en el servidor con el adaptador @astrojs/node en modo standalone, las páginas consultan PostgreSQL en el servidor y devuelven HTML, GET /api/health es la única ruta JSON, los formularios HTML envían solicitudes GET o POST, app/src/lib/db.js mantiene un pg.Pool compartido y las consultas usan parámetros, la creación y edición de un evento guardan el registro principal y sus detalles dentro de una transacción.

**Flujode:** navegador → páginas y formularios Astro → capa de datos (app/src/lib/) → PostgreSQL.




**En resumen; I**la persona envía una solicitud a Astro, Astro consulta o actualiza PostgreSQL mediante app/src/lib/ y responde HTML o JSON, el diagrama resume el flujo observado en el código; no representa servicios externos.

**Estructura**

| Ruta | Responsabilidad |
|---|---|
| app/src/pages/ | Resumen, CRUD de módulos y endpoint de saludo |
| app/src/components/ | Formularios compartidos, alertas y controles de eventos del sistema |
| app/src/layouts/Layout.astro | Navegación y estilos globales |
| app/src/lib/ | db.js abre el pool y las transacciones, datos-evento.js consulta catálogos, eventos.js guarda eventos e inventario, validaciones.js, formularios.js y errores.js concentran validación y mensajes |
| database/ddl/ | Creación del esquema de la BD y permisos del rol de aplicación |
| database/datos/ | Datos de muestra(prueba) |
| database/consultas/ | Consultas de demostración iniciales |
| database/pruebas-integridad/ | Casos SQL de integridad de entidad, referencial y dominio |
| app/.env.example | Plantilla sin credenciales reales para configurar “DATABASE_URL” |
| app/dist/ | Salida generada por npm run build, se puede volver a crear y Git la ignoraría |
| docs/ y evidencias/ | Manuales, requerimientos y material de respaldo del proyecto |

**Requisitos, configuración y puesta en marcha**

Se requiere Node.js 22.12 o posterior, npm, PostgreSQL y el cliente psql, la versión mínima de Node está declarada en app/package.json, a base de datos debe existir antes de cargar el esquema, separa el rol propietario de la base del rol eventos_app que usa la aplicación.

En una instalación local, crea la base y el rol con un usuario administrador de PostgreSQL, en psql, establece la contraseña de forma interactiva para evitar escribirla en el historial:

```
createdb eventos
psql -d postgres
CREATE ROLE eventos_app LOGIN;
\password eventos_app
\q
```

Desde la raíz del proyecto, aplica el esquema y los datos con una conexión de propietario o administrador, seemplaza los valores de ejemplo, luego codifica los caracteres reservados de una contraseña al incorporarla en una URI, así.

```
export DATABASE_URL_ADMIN=postgresql://administrador:contrasena@localhost:5432/
eventos
psql "$DATABASE_URL_ADMIN" -v ON_ERROR_STOP=1 -f database/ddl/01_ddl.sql
psql "$DATABASE_URL_ADMIN" -v ON_ERROR_STOP=1 -f
database/datos/01_datos_prueba.sql
psql "$DATABASE_URL_ADMIN" -v ON_ERROR_STOP=1 -f database/ddl/02_permisos.sql
```

**Advertencia destructiva:** 01_ddl.sql elimina y vuelve a crear las seis tablas del sistema, se pierden sus registros, se usa para preparar o restablecer una base desechable, no debe de ejecutarse para actualizar una base con datos que deban conservarse, el script de permisos concede USAGE en public, DML sobre las tablas existentes y acceso a sus secuencias, no crea el rol ni concede permiso para crear tablas.

Configura app/.env a partir de la plantilla y usa el rol de aplicación, asi:

```
cd app
cp .env.example .env
Edita .env y asigna
DATABASE_URL=postgresql://eventos_app:contrasena@localhost:5432/eventos.
```

El archivo .env está excluido de Git, no se copian las credenciales reales a ejemplos, capturas ni manuales.

Instala dependencias y levanta el servidor de desarrollo desde app/:

```
npm ci
npm run dev
```

Astro sirve la aplicación para desarrollo, normalmente en http://localhost:4321, detén el proceso con Ctrl+C. npm run build compila y genera dist/; no levanta un servidor, para revisar localmente esa compilación, ejecuta npm run preview y detén el proceso con Ctrl+C.

La aplicación usa output: server y el adaptador Node en modo standalone, para ejecutar directamente la salida de producción, primero compila con DATABASE_URL disponible y luego inicia el punto de entrada desde app/:

```
npm run build
HOST=127.0.0.1 PORT=8080 node
./dist/server/entry.mjs
```

El adaptador lee HOST y PORT al iniciar; define HOST=0.0.0.0 solo cuando el despliegue necesita escuchar en interfaces externas y cuenta con controles de red adecuados, el proceso se detiene con Ctrl+C o mediante el administrador de servicios usado por el entorno.

**Limitación de configuración:** app/src/lib/db.js usa import.meta.env.DATABASE_URL, Astro incorpora ese valor al compilar el servidor, por ello, dist/ queda ligado a la URI disponible durante npm run build, cambiar solo la variable al iniciar no sustituye el valor compilado, trata dist/ como material sensible y no lo publiques si contiene una credencial real, para desplegar con secretos inyectados al iniciar, cambia la lectura a una variable de entorno de runtime y vuelve a compilar.

Después del arranque, comprueba la conexión con:

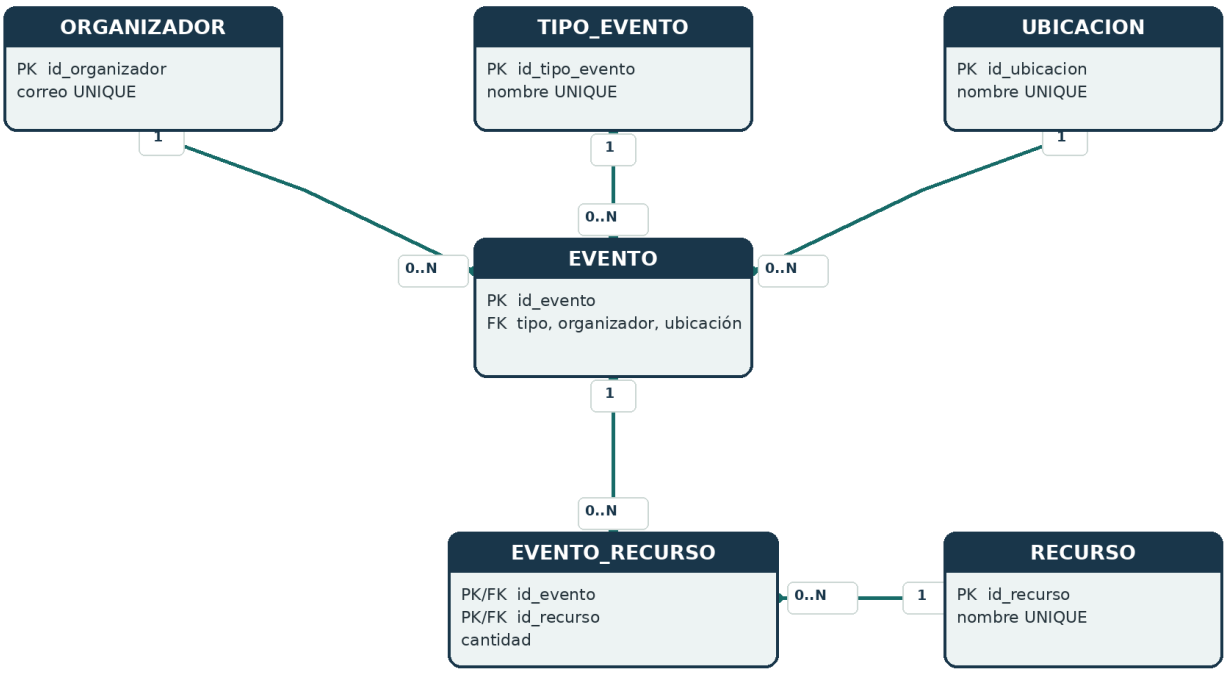
```
curl -i http://127.0.0.1:8080/api/health
```

Un 200 con {"estado":"ok"} confirma que el servidor puede ejecutar SELECT 1. Esta prueba no verifica los permisos sobre todas las tablas usadas por las páginas.

**Modelo entidad-relación**

El siguiente diagrama refleja el DDL PostgreSQL actual, cada evento tiene un organizador, un tipo y una ubicación; puede tener cero o más detalles de recursos, la tabla EVENTO_RECURSO resuelve la relación muchos-a-muchos entre eventos y recursos.

### Modelo entidad-relación




PK: clave primaria · FK: clave foránea · 1 a 0..N: una fila puede relacionarse con cero o muchas.

**Resumen:** cinco entidades principales se conectan mediante EVENTO; EVENTO_RECURSO enlaza cada evento con los recursos asignados, los tres catálogos y RECURSO pueden existir sin registros de evento.

#### Entidades, atributos y restricciones

Los tipos y restricciones siguientes vienen de database/ddl/01_ddl.sql. SERIAL crea una clave entera respaldada por secuencia.

| Entidad | Atributos y reglas |
|---|---|
| ORGANIZADOR | id_organizador SERIAL PK; nombre VARCHAR(80) y apellido VARCHAR(80) NOT NULL; correo VARCHAR(120) NOT NULL UNIQUE; telefono VARCHAR(20) y cargo VARCHAR(80) opcionales. |
| TIPO_EVENTO | id_tipo_evento SERIAL PK; nombre VARCHAR(50) NOT NULL UNIQUE; descripcion VARCHAR(200) opcional. |
| UBICACION | id_ubicacion SERIAL PK; nombre VARCHAR(80) UNIQUE y direccion VARCHAR(150) NOT NULL; capacidad INTEGER NOT NULL y mayor que cero. |

| Entidad | Atributos y reglas |
|---|---|
| RECURSO | id_recurso SERIAL PK; nombre VARCHAR(80) NOT NULL UNIQUE; descripcion VARCHAR(200) opcional; cantidad_total INTEGER NOT NULL y mayor que cero; disponible BOOLEAN NOT NULL, valor predeterminado TRUE. |
| EVENTO | id_evento SERIAL PK; nombre VARCHAR(120) NOT NULL; descripcion VARCHAR(300) opcional; fecha DATE, hora_inicio TIME y estado VARCHAR(20) NOT NULL; hora_fin TIME opcional; tres FK obligatorias a tipo, organizador y ubicación. hora_fin debe ser posterior a hora_inicio cuando se indica. estado admite Programado, En curso, Finalizado o Cancelado. |
| EVENTO_RECURSO | FK id_evento INTEGER y id_recurso INTEGER, PK compuesta por ambas; cantidad INTEGER NOT NULL y mayor que cero. Una combinación evento-recurso aparece como máximo una vez. |

**Relaciones y borrado**

| Relación | Cardinalidad | Regla de borrado |
|---|---|---|
| Organizador–evento | Un organizador puede tener cero o muchos eventos, cada evento tiene exactamente uno | RESTRICT: no borrar organizador en uso |
| Tipo–evento | Un tipo puede clasificar cero o muchos eventos, cada evento tiene exactamente uno | RESTRICT |
| Ubicación–evento | Una ubicación puede alojar cero o muchos eventos, cada evento tiene exactamente una | RESTRICT |

| Relación | Cardinalidad | Regla de borrado |
|---|---|---|
| Evento–detalle | Un evento puede tener cero o muchos detalles, cada detalle pertenece a un evento | CASCADE: al borrar el evento, se borran sus detalles |
| Recurso–detalle | Un recurso puede aparecer en cero o muchos detalles, cada detalle refiere a un recurso | RESTRICT: no borrar un recurso asignado |

**Normalización hasta tercera forma normal**

- **Primera forma normal (1FN):** cada columna guarda un valor por fila, la lista repetible de recursos de un evento se separa en filas de EVENTO_RECURSO, no se almacena como texto delimitado en EVENTO.
- **Segunda forma normal (2FN):** EVENTO_RECURSO usa la PK compuesta (id_evento, id_recurso). La cantidad asignada depende de la combinación completa, los atributos del evento y del recurso viven en sus respectivas tablas, no dependen de solo una parte de esa clave compuesta.
- **Tercera forma normal (3FN):** los datos descriptivos de organizadores, tipos, ubicaciones y recursos se guardan una vez en sus catálogos, EVENTO conserva sus identificadores como FK, sin repetir correo, nombre de tipo o dirección, los atributos no clave describen la clave de su propia tabla y no otro atributo no clave.

**Rutas y contratos**

| Método y ruta | Comportamiento |
|---|---|
| GET / | Resumen de conteos y agenda próxima |
| GET /eventos | Listado con desde, hasta, tipo y proximos=1, fechas inclusivas y próximos desde CURRENT_DATE, sin cancelados |
| GET/POST /eventos/nuevo | Muestra el formulario y crea el evento con sus detalles de recursos |
| GET /eventos/:id | Muestra el detalle del evento. Un ID inexistente responde 404 |
| POST /eventos/:id | Elimina el evento cuando el formulario envía accion=eliminar, ON DELETE CASCADE elimina sus detalles |
| GET/POST /eventos/:id/editar | Consulta y actualiza el evento, sustituye sus recursos dentro de una transacción |
| GET /organizadores, /tipos-evento, /ubicaciones, /recursos | Muestra los listados. GET /recursos?fecha=AAAA-MM-DD calcula compromisos para esa fecha |
| GET/POST /<catalogo>/nuevo | Muestra y procesa el alta en el catálogo |

| Método y ruta | Comportamiento |
|---|---|
| | solicitado |
| GET/POST /<catalogo>/:id/editar | Muestra y procesa la edición del registro indicado |
| POST /<catalogo> | Elimina cuando el formulario envía accion=eliminar, una FK puede impedir el borrado. |
| GET /api/health | 200 {"estado":"ok"} si SELECT 1 funciona; 503 {"estado":"sin conexión a la base de datos"} si falla |

Los formularios de alta, edición y borrado usan POST, las escrituras exitosas responden con redirección 303 para volver al listado o detalle. No hay una API REST para los catálogos. No existe autenticación ni autorización de usuario; tampoco se requiere autenticación para el endpoint de salud.

### Lógica de recursos y transacciones

app/src/lib/eventos.js ejecuta la creación o actualización del registro principal y del detalle dentro de withTransaction, antes de guardar cantidades, bloquea con FOR UPDATE los recursos seleccionados y calcula las unidades ya comprometidas en la misma fecha por eventos que no están cancelados, si la cantidad solicitada excede lo disponible, hace ROLLBACK y devuelve el error; al editar excluye el mismo evento del cálculo, poner cantidad 0 omite la fila de detalle, durante una edición el conjunto anterior se sustituye por el enviado.

La disponibilidad es por fecha, no por intervalo de horas, el formulario no recalcula el indicador en el navegador cuando cambia la fecha, aunque el servidor siempre vuelve a validar al guardar, el DDL no impone conflictos de horario entre eventos de una misma ubicación ni compara capacidad con aforo, esas reglas no están implementadas.

### Errores visibles

app/src/lib/errores.js traduce SQLSTATE comunes; app/src/components/Alert.astro también presenta el detalle técnico de PostgreSQL, entre ellos están 23505 (valor único repetido), 23503 (clave foránea), 23502 (campo obligatorio), 23514 (regla CHECK), 22001 (texto largo) y 22007/22008 (fecha u hora inválida/fuera de rango), en producción, oculta el detalle SQL al usuario final, si necesitas diagnósticos, guárdalos en registros protegidos del servidor.

Los atributos required, los límites de texto y el formato de correo se validan en el navegador, no equivalen a una validación completa del servidor. PostgreSQL protege los campos obligatorios, largos, únicos y las reglas CHECK/FK. El DDL no valida el formato del correo, las solicitudes enviadas por fuera del navegador dependen de las restricciones de la base y de las comprobaciones específicas de app/src/lib/validaciones.js y app/src/lib/eventos.js.

### Pruebas e integridad

No hay un ejecutor de pruebas JavaScript configurado en app/package.json, los scripts de database/pruebas-integridad/ cubren integridad de entidad, referencial y dominio, ejecútalos en orden solo después de cargar el DDL y database/datos/01_datos_prueba.sql en una base desechable.

```
psql "$DATABASE_URL_ADMIN" -f
database/pruebas-integridad/01_integridad_entidad.sql
psql "$DATABASE_URL_ADMIN" -f
database/pruebas-integridad/02_integridad_referencial.sql
psql "$DATABASE_URL_ADMIN" -f
database/pruebas-integridad/03_integridad_dominio.sql
```

Ejecuta los scripts en orden y no añadas -v ON_ERROR_STOP=1 a esos tres comandos las sentencias marcadas DEBEN FALLAR generan errores esperados y el guion continúa con los siguientes casos, las operaciones permitidas se ejecutan con ROLLBACK, revisa las salidas y el conteo final esperado: 10 organizadores, 5 tipos, 6 ubicaciones, 10 recursos, 10 eventos y 30 detalles. El código de salida de psql por sí solo no interpreta los casos esperados.

**Seguridad y operación**

- Mantén DATABASE_URL fuera del repositorio y usa eventos_app con permisos mínimos. 02_permisos.sql concede DML y acceso a secuencias a las tablas existentes; no crea el rol ni permisos para crear el esquema. Repite los GRANT al incorporar objetos.
- La aplicación no incluye autenticación, autorización ni una verificación CSRF propia. Limita su exposición a una red confiable hasta implementar esos controles.
- Los errores actuales pueden mostrar mensajes, detalles y nombres de restricciones de PostgreSQL, oculta esa información al usuario final en producción.
- Conserva respaldos antes de cambiar el esquema o desplegar una versión que altere datos el repositorio no implementa migraciones ni respaldo automático.

**Respaldo y restauración**

El proyecto no crea copias automáticas. Antes de modificar el esquema o actualizar una base con información que deba conservarse, genera un respaldo custom de PostgreSQL, el archivo puede contener correos, teléfonos y otros datos; guárdalo fuera de Git y limita el acceso.

```
pg_dump --format=custom --file=./eventos-respaldo.dump "$DATABASE_URL_ADMIN"
pg_restore --list ./eventos-respaldo.dump
```

Prueba la recuperación primero en una base nueva, define DATABASE_URL_RESTAURACION con la conexión del propietario y aplica los permisos después de restaurar:

```
createdb eventos_recuperacion
export
DATABASE_URL_RESTAURACION=postgresql://administrador:contrasena@localhost:5432/
eventos_recuperacion
pg_restore --dbname="$DATABASE_URL_RESTAURACION" --no-owner --no-acl ./eventos-
respaldo.dump
psql "$DATABASE_URL_RESTAURACION" -v ON_ERROR_STOP=1 -f
database/ddl/02_permisos.sql
```

Verifica los conteos y ejecuta las comprobaciones de integridad antes de cambiar la conexión de la aplicación, pg_restore no necesita ejecutar 01_ddl.sql, el respaldo ya contiene el esquema, no restaures sobre la base activa sin un procedimiento de reemplazo y reversión probado.

### Diagnóstico operativo

| Síntoma | Comprobaciones y acción |
|---|---|
| /api/health responde 503. | Comprueba PostgreSQL, la URI incorporada al build, host, base y credenciales, si cambiaste .env, vuelve a compilar |
| Salud 200, pero una página falla. | La ruta solo ejecuta SELECT 1. Verifica que el DDL se cargó en esa base y que eventos_app tiene permisos sobre tablas y secuencias |
| permission denied en tabla o secuencia. | Ejecuta 02_permisos.sql contra la base correcta con un propietario o administrador, repite los GRANT al crear objetos. |
| SQLSTATE 23505 o 23503. | 23505 indica duplicado en un campo único; 23503 indica una relación que impide la operación, corrige el dato o conserva el registro referenciado |
| Puerto ocupado al iniciar. | Cambia PORT a un puerto disponible y vuelve a ejecutar el punto de entrada |
| PostgreSQL rechaza fecha u hora. | Usa AAAA-MM-DD, confirma que la fecha exista y que hora_fin sea posterior a hora_inicio cuando se proporciona |

El endpoint de saludo confirma disponibilidad de conexión, no la corrección del esquema ni los permisos de cada pantalla, al compartir diagnósticos, elimina URI, contraseñas, correos y teléfonos de las salidas.

### Cambios de esquema y mantenimiento

El repositorio no contiene un ejecutor de migraciones. 01_ddl.sql reconstruye el esquema completo y no es una migración incremental, para cambiar el modelo:

1. Respaldar la base y reproducir el cambio en una base local desechable.
2. Añadir un script SQL incremental y descriptivo bajo database/ddl/, no reemplazar el respaldo con un DROP TABLE.
3. Actualizar la aplicación, los datos de ejemplo y los casos afectados en database/pruebas-integridad/.
4. Aplicar el script en una copia del esquema, revisar restricciones y probar la restauración.
5. Ejecutar npm ci y npm run build desde app/, aplicar el cambio a la base activa solo con respaldo y reversión disponibles.

Después de crear tablas o secuencias, revisa los permisos de eventos_app; 02_permisos.sql otorga permisos sobre objetos existentes, mantener package-lock.json junto con cambios de dependencias, no hay scripts de lint ni pruebas JavaScript en app/package.json.

### Convenciones de código

Las páginas Astro van en minúsculas bajo app/src/pages/, los componentes Astro usan PascalCase, por ejemplo EventoForm.astro. JavaScript usa módulos ES, sangría de dos espacios y nombres camelCase, los identificadores SQL usan snake_case en minúsculas, parametriza toda consulta con datos externos ($1, $2, …) no concatene valores de formularios en una sentencia SQL.

## Lista de comprobación para desplegar

- [ ] La base de destino se creó y el esquema se aplicó con un usuario propietario.
- [ ] El rol eventos_app existe y tiene permisos DML y de secuencias.
- [ ] DATABASE_URL identifica la base correcta y usa una credencial con permisos mínimos.
- [ ] La variable estaba disponible al ejecutar npm run build; dist/ permanece privado mientras incluya esa URI.
- [ ] npm run build terminó y el servidor standalone inició con HOST y PORT previstos.
- [ ] GET /api/health devuelve 200 y se probó una pantalla que consulta tablas.
- [ ] El servicio está restringido a una red autorizada mientras no exista autenticación.
- [ ] Existe un respaldo recuperable y no se distribuyen .env, dist/ ni copias con datos personales.

## Referencias de mantenimiento

- Aplicación: configuración y comandos · página principal · rutas de eventos
- Flujo de eventos: formulario · transacciones y recursos · manejo de errores
- Base de datos: esquema · permisos del rol · consultas de demostración
- Proyecto: requerimientos